LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Progress meeting 2: from plan to executed system, found bugs and measured results

Continues the meeting of 16 October 2026

CS-5187 term project A3 · Venu · Bhuvan · Gowtham · 29 October 2026

Where we were on 16 October

The question (meeting 1)

Can a typed, editable grammar, extended by an LLM that reads PostgreSQL's own commits, find more bugs than random mutation and a plain grammar, under the same budget?

State on 16 Oct

- Harness, grammar and random baseline built
- 100% valid on 300 cases, 0 bug candidates
- No campaign run yet; LLM loop only planned

Our three claims

- 1. One tree, four jobs: the derivation tree generates, mutates, is rewritten by oracles, and is reduced. SQL text is only its rendering.
- 2. Soundness we measure: generator validity and oracle precision are reported numbers, not assumptions.
- 3. The LLM proposes, the server decides: a proposal is used only if it runs and reaches the changed lines.

So what / next: Everything today answers one question: did the three claims survive contact with a real server?

What we promised, and where each promise stands

Meeting 1 said	Now (29 Oct)	
Typed grammar, ~100 rules, 3 oracles	78 rules / 421 alternatives; 12 oracle kinds, 18 plan variants, cross-version	done
LLM loop: mine, propose, gate	Built; run on 10 commits; 4 accepted by the gate	done
8-12 re-injected historical bugs	23 reverted fixes, 20 observable, 3 server variants each	exceeded
10 trials per arm	5 trials x 1 h x 7 arms on two machines (time budget)	reduced
Gate: >= 80% valid, 200 samples	Gate: >= 60% valid, 150 samples (calibrated on the server)	changed
Target: one bug acknowledged upstream	4 new bugs, 2 confirmed by committers, 1 fixed on master	met
Bug candidates: 0 so far	93 candidates triaged, each with a verdict	done

So what / next: Each row is a chapter of today's story; the next slide shows the order.

The story of this talk



typed grammar, tree, oracles

first real bugs, upstream

reduce, de-duplicate, precision

commit-directed extension

benchmark and campaigns

RQ1 to RQ4, lessons

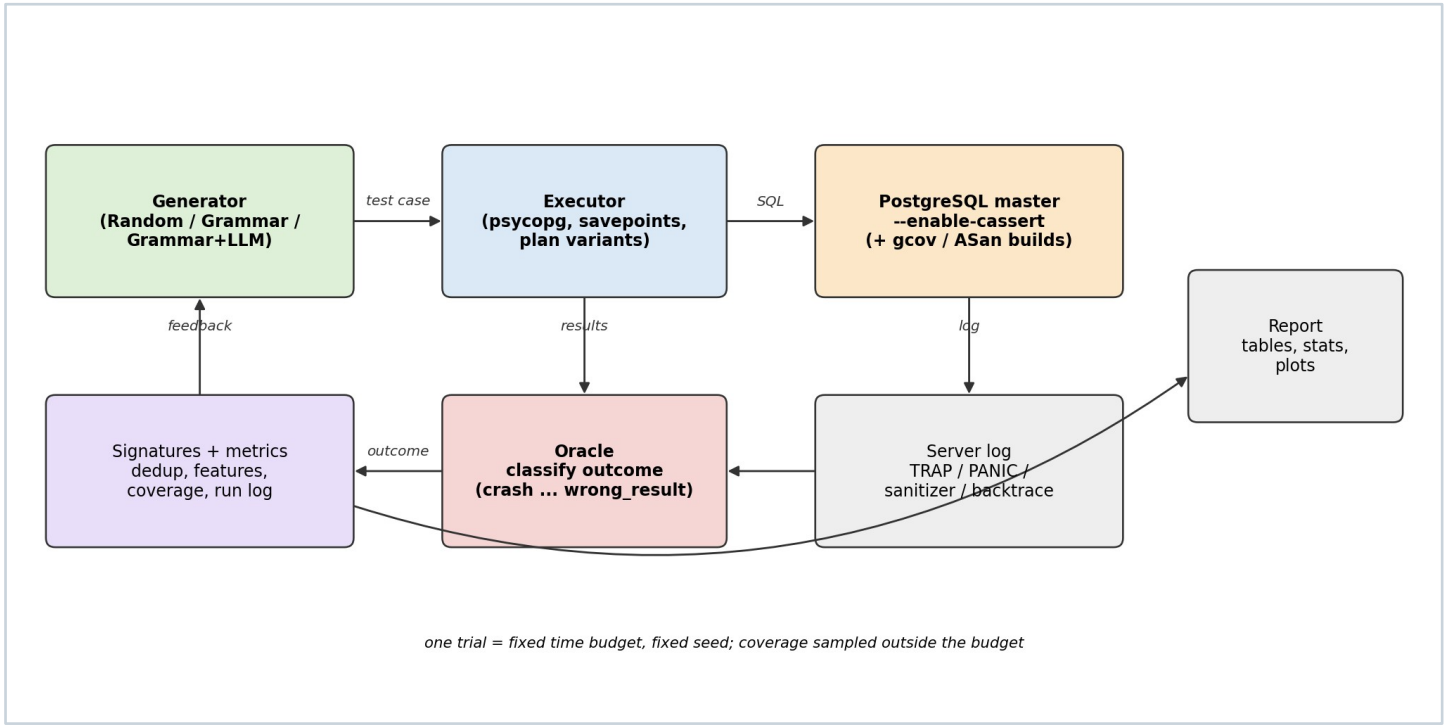
plan to 13 Nov

The running thread

The fuzzer generates SQL from a typed tree. Oracles turn the tree into checks that detect wrong results, not only crashes. Every failure is reduced and verified, because an unverified bug report wastes the maintainers' time. The LLM widens what the tree can say. The benchmark and campaigns measure whether each part helps.

So what / next: Build the system, use it to find bugs, make sure the findings are trustworthy, add the LLM, then measure everything.

Build 1: a typed grammar that keeps the derivation tree



generator, executor, instrumented server, oracle, report (as at meeting 1, now complete)

78 rules · 421 alternatives · 13 types · 9-relation schema (partitions, inheritance, view, 4 index types)

In words

- SQL is generated as a tree, not as text. Each node knows its type and scope.
- A node can be replaced by another derivation of the same type, so mutated queries stay valid SQL.
- Weights learn from feedback: alternatives that find new plans or features are chosen more often.
- Literals come from the data (min, max, quantiles, NaN, -0) so predicates select real rows.

So what / next: A tree with typed, positioned nodes is what makes safe rewriting possible, which is the next slide.

Build 2: oracles that know the right answer without a reference

The idea (ternary logic partitioning)

- Q: SELECT … WHERE p
- Every row makes p true, false or NULL: exactly one
- Q must equal Q(p) UNION ALL Q(NOT p) UNION ALL Q(p IS NULL)
- Different rows = bug in the planner or executor

```
SELECT c, '(,)'::int4range @> c  -- must be NULL for NULL
c
FROM (VALUES (NULL::int), (1)) v(c);

-- 17, 18.6, master: t   (planner folded it)
-- e vs (SELECT e) disagree  =>  bug
```

Oracle family	Compares
TLP (WHERE, HAVING, aggregates), NoREC	query vs partitioned / counted form
Expression and predicate rewrites	e vs (SELECT e), COALESCE(e,e), NOT NOT p …
RLS leak check	restricted role vs superuser with policy inlined
18 plan variants	same query, different planner settings
Cross-version	master vs release 18.6
Crash / assertion / internal error	server log and error class

So what / next: Oracles need sound rewrites; whenever one breaks, we get a false positive. Hold that thought for the triage chapter.

Find 1: the first real bug, found by an oracle

```
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ PSQL=~/pgfuzz/install/REL_18_STABLE/bin/psql
for p in 5435 5440; do
  $PSQL -h 127.0.0.1 -p $p -U postgres \
    -c "SELECT version()" \
    -c "SELECT c, '(,)'::int4range @> c AS a, 'empty'::int4range @> c AS b, NOT ('empty'::int4range @> c) AS d FROM (VALUES (NULL::int), (1)) v(c);" \
    -c "EXPLAIN VERBOSE SELECT '(,)'::int4range @> c FROM (VALUES (NULL::int), (1)) v(c);"
done
                                                  version

--------------------------------------------------------------------------------
--------------------------
 PostgreSQL 18.6 on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubunt
u2~24.04.1) 13.3.0, 64-bit
(1 row)

 c | a | b | d
---+---+---+---
   | t | f | t
 1 | t | f | t
(2 rows)

                        QUERY PLAN
-----------------------------------------------------------
 Values Scan on "*VALUES*"  (cost=0.00..0.03 rows=2 width=1)
   Output: true
(2 rows)

                                                  version

--------------------------------------------------------------------------------
-----------------------------
 PostgreSQL 20devel on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ub
untu2~24.04.1) 13.3.0, 64-bit
(1 row)

 c | a | b | d
---+---+---+---
   | t | f | t
 1 | t | f | t
(2 rows)

                        QUERY PLAN
-----------------------------------------------------------
 Values Scan on "*VALUES*"  (cost=0.00..0.03 rows=2 width=1)
   Output: true
(2 rows)
```

What you see

- NULL row should give NULL in every column
- 18.6 and master print true
- EXPLAIN shows Output: true: folded to a constant

Why only an oracle finds it

- No crash, no error: the result is just wrong
- Present in released 17 and 18
- Root cause: a planner support function in rangetypes.c; patch sent for 17, 18 and master

So what / next: One bug is anecdote; the next slide shows how many we found and where each stands upstream.

Find 2: from first find to nine upstream reports in two weeks

Date	Bug	Found by	Upstream
16 Oct	BUG-01 JSON_TABLE collations	grammar fuzzer (assertion)	#19768 filed
23 Oct	BUG-02 range containment NULL	scalar-subquery rewrite oracle	#19793 filed + patch
23 Oct	AUDIT-F1 NOT IN anti-join cast; AUDIT-F2 IGNORE NULLS hang	code audit + probes	pgsql-bugs; F2 answered
27 Oct	C-01 dependencies.c assertion	differential fuzzers	pgsql-bugs filed
28 Oct	#19802 MERGE SubPlan error	grammar campaign	**confirmed, patch posted**
28 Oct	#19803 parallel Append assertion	LLM-fragment arm + TLP	**confirmed, fixed on master**
Also	#19771 to_char NaN crash	grammar campaign	**fix committed (Gowtham)**

So what / next: Finding is only half: each report needs a verified reproducer, so every candidate goes through triage.

Find 3: upstream scorecard

Member	Bugs filed	Replied: confirmed a bug	Fixed upstream
Venu	2	0	0
Gowtham	3	**3**	**2**
Bhuvan	4	1	0
Team total	**9**	**4**	**2**

9

reports on pgsql-bugs / pgsql-hackers

4

answered by committers or contributors

2

fixed on PostgreSQL master

So what / next: Nine reports means nine reproducers to defend; the next stage is how we make them trustworthy.

Bugs: Venu (2 filed)

Bug	Upstream	What it is	Status
BUG-01 JSON_TABLE collations	#19768	JSON_TABLE never assigns collations to its context item and PASSING arguments. Assertion in execExpr.c on master; silent wrong result on 18.6. One-line patch + tests.	**filed 16 Oct; no reply**
BUG-02 range containment with NULL	#19793	Planner support function folds range @> NULL element to a constant. Silent wrong result in released 17 and 18.	**filed 23 Oct; patch sent; no reply**

So what / next: BUG-01 came from a campaign assertion; BUG-02 from an oracle. Both have patches and tests.

Bugs: Gowtham (3 filed, 3 answered, 2 fixed)

Bug	Upstream	What it is	Status
MERGE: SubPlan with no parent plan	#19802	MERGE into a partitioned table with a non-pulled-up source and a scalar subquery in UPDATE SET fails with an internal error.	**confirmed; fix posted**
Parallel Append assertion	#19803	Assertion failure in a parallel Append plan, found through a TLP check by the LLM-fragment arm.	**confirmed; fixed (abda2a2c)**
to_char NaN / NORMALIZE crash	#19771	to_char on NaN leads to an assertion in NORMALIZE; reduced by our tool to 27 tokens.	**fix committed (776df8a6)**

So what / next: Two of these came directly from the campaigns of this meeting's evaluation.

Bugs: Bhuvan (4 filed, 1 answered)

Bug	Upstream	What it is	Status
AUDIT-F1 NOT IN anti-join cast	pgsql-bugs	Wrong-result regression from commit 383eb21ebf: NOT IN becomes an anti-join and ignores an implicit cast. In 19rc1 and master, not 18.6. Patch + test.	filed 23 Oct
AUDIT-F2 IGNORE NULLS hang	pgsql-bugs	Frame scan of window functions with IGNORE NULLS can hang.	filed 23 Oct; Ishii replied
C-01 dependencies.c assertion	pgsql-bugs	Duplicate expression in extended statistics trips an assertion. Reproducer + patch.	filed 27 Oct
Extended stats non-finite MCV	pgsql-hackers	pg_restore_extended_stats accepts non-finite MCV frequencies. Patch submitted.	patch under review

So what / next: Bhuvan's bugs come from auditing recent PostgreSQL code with differential probes, a second route to bugs next to the fuzzer.

Triage 1: every finding is reproduced, shrunk and localized

```
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ cat ~/cs5187-term-project/runs/memberA-rq3/logs/A3_triage.txt
[cde9712bc8db] assertion via assertion: reproducing x3
  statements: 3 -> 2
  statements: 2 -> 1
  clauses: 572 -> 570 tokens
  clauses: 570 -> 543 tokens
  clauses: 543 -> 529 tokens
  clauses: 529 -> 527 tokens
  clauses: 527 -> 515 tokens
  items: 515 -> 420 tokens
  items: 420 -> 410 tokens
  items: 410 -> 318 tokens
  items: 318 -> 314 tokens
  items: 314 -> 290 tokens
  items: 290 -> 277 tokens
  items: 277 -> 273 tokens
  items: 273 -> 269 tokens
  items: 269 -> 265 tokens
  items: 265 -> 263 tokens
  items: 263 -> 261 tokens
  groups: 261 -> 201 tokens
  groups: 201 -> 162 tokens
  groups: 162 -> 141 tokens
  groups: 141 -> 139 tokens
  groups: 139 -> 125 tokens
  groups: 125 -> 116 tokens
  reduced to 1 statement(s), 116 tokens in the culprit (406 runs, 73.3 s)
1 candidate(s) triaged into ../results/triage_demo; verdict template: ../results/triage_demo/verdicts.template.yaml
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$
```

pgfuzz triage on a real candidate

```
bhuvi@BlazingBhuvi:/mnt/c/Users/Bhuva/OneDrive/Desktop/cs5187-term-project-main$ cat ~/cs5187-term-project/runs/memberA-rq3/triage_demo/cde9712bc8db/upstream_report
.txt 2>&1
Subject: fails an assertion -- <one line: the feature involved>

PostgreSQL version: 20devel (built with --enable-cassert)
Also checked on: <latest release: run the script there too>
Platform: <OS, compiler>

Description
-----------

The following script fails an assertion.

TRAP: failed Assert("list_length(rcexpr->inputcollids) == nopers"), File: "execExpr.c", Line: 2078

Script
------

-- pgfuzz triage of cde9712bc8db: assertion via assertion
SELECT CASE CASE WHEN NULL THEN NULL ELSE (SELECT MAX ('{NULL}'::text[]) FROM JSON_TABLE (CASE WHEN (ROW (10.0::numeric, 10) >= ROW (0.0::numeric, 1) IS FALSE) THEN
 '{"k": -1, "a": [1, 2, 3]}'::jsonb ELSE '{"k": 1, "a": [1, 2]}'::jsonb END, '$.a[*]' COLUMNS ( c1 int PATH '$')) AS a6) END WHEN NULL THEN NULL WHEN MAX ('{a,b,c,d
}'::text[]) OVER (PARTITION BY '{1,2,3}'::int[]) THEN NULL ELSE '{NULL,1}'::int[] END AS c0 FROM (SELECT a1.c0 AS c0 FROM t0 AS a1) AS a2;

Expected
--------

<what should happen, and why: SQL standard / documentation section>

<If bisected: first bad commit (scripts/bisect_pg.sh).>
bhuvi@BlazingBhuvi:/mnt/c/Users/Bhuva/OneDrive/Desktop/cs5187-term-project-main$
```

generated upstream report draft

590 → 79

tokens after tree + token reduction of the BUG-01 case (tokens alone: 116)

x3

reproduced three times on a fresh connection

localized

variant removes it: planner path; none: always-applied rewrite

So what / next: Reduction gives the maintainers a small reproducer; the remaining question is how many findings were real.

Triage 2: the price of strong oracles is false positives

Verdict	Count
real, new	7
duplicate of a real upstream bug	15
false positive	37
flaky (not reproducible x3)	28
open, to verify	6

0.16

precision, duplicates not counted

0.37

if duplicates count

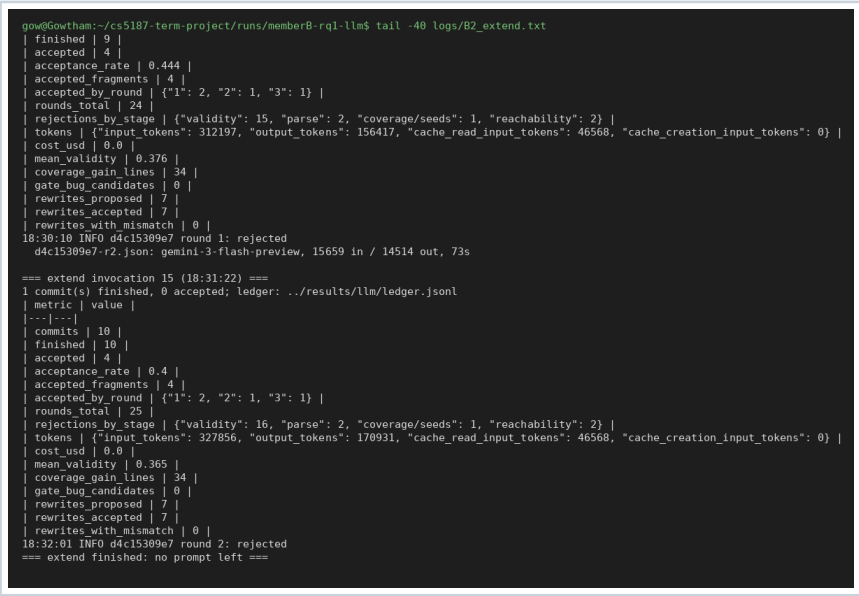
10 false-positive classes, all fixed with tests

- 6 during development (ARRAY(SELECT), TLP-HAVING, numeric scale, seed clock, PoC replay, an intended change)
- Equal intervals printed differently: 16 candidates
- ORDER BY NULL in ordered aggregates: 2
- Cross-version state divergence: 5
- LLM rewrite that changed the collation: 1

So what / next: Precision turned out to be the limiting factor, not generator power. This shaped the rest of the work.

LLM: proposals only count if the server agrees

Mine commits → LLM proposes → **Gate on server** → Repair (max 3) → Enters the campaign



```
gow@Gowtham:~/cs5187-term-project/runs/memberB-rq1-llm$ tail -40 logs/B2_extend.txt
| finished | 9 |
| accepted | 4 |
| acceptance_rate | 0.444 |
| accepted_fragments | 4 |
| accepted_by_round | {"1": 2, "2": 1, "3": 1} |
| rounds_total | 24 |
| rejections_by_stage | {"validity": 15, "parse": 2, "coverage/seeds": 1, "reachability": 2} |
| tokens | {"input_tokens": 312197, "output_tokens": 156417, "cache_read_input_tokens": 46568, "cache_creation_input_tokens": 0} |
| cost_usd | 0.0 |
| mean_validity | 0.376 |
| coverage_gain_lines | 34 |
| gate_bug_candidates | 0 |
| rewrites_proposed | 7 |
| rewrites_accepted | 7 |
| rewrites_with_mismatch | 0 |
18:30:10 INFO d4c15309e7 round 1: rejected
  d4c15309e7-r2.json: gemini-3-flash-preview, 15659 in / 14514 out, 73s

=== extend invocation 15 (18:31:22) ===
1 commit(s) finished, 0 accepted; ledger: ../results/llm/ledger.jsonl
| metric | value |
|---|---|
| commits | 10 |
| finished | 10 |
| accepted | 4 |
| acceptance_rate | 0.4 |
| accepted_fragments | 4 |
| accepted_by_round | {"1": 2, "2": 1, "3": 1} |
| rounds_total | 25 |
| rejections_by_stage | {"validity": 16, "parse": 2, "coverage/seeds": 1, "reachability": 2} |
| tokens | {"input_tokens": 327856, "output_tokens": 170931, "cache_read_input_tokens": 46568, "cache_creation_input_tokens": 0} |
| cost_usd | 0.0 |
| mean_validity | 0.365 |
| coverage_gain_lines | 34 |
| gate_bug_candidates | 0 |
| rewrites_proposed | 7 |
| rewrites_accepted | 7 |
| rewrites_with_mismatch | 0 |
18:32:01 INFO d4c15309e7 round 2: rejected
=== extend finished: no prompt left ===
```

commit-directed run (Gemini Flash)

Gate checks

- Fragment parses
- Used >= 5 times in 150 cases
- >= 60% of statements run
- Seeds run
- More changed lines covered

Outcome

- 10 commits, 4 accepted
- 16 of 25 rounds failed on validity
- 7 rewrites passed; 1 changed the collation, gate now rejects COLLATE
- 0 USD (free tier)

So what / next: The gate worked as a filter but the model was modest; the next chapter measures what it added.

Measure 1: ground truth from 23 real, recently fixed bugs

```
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ python3 -m pgfuzz bench verify configs/rq3.yaml
| bug | commit | symptom | all injected | evidence |
|---|---|---|---|---|
| rb-11b56ee2 | 11b56ee2 | assertion | assertion | pathkeys.c: var->varno == rel->relid |
| rb-94e1385e | 94e1385e | wrong_result | wrong_result | stmt 0: 1 rows on fixed, 1 on buggy |
| rb-41180127 | 41180127 | missing_error | missing_error | stmt 0: fixed raises 22030 duplicate JSON object key value |
| rb-b28a2b2b | b28a2b2b | wrong_result | wrong_result | stmt 5: 1 rows on fixed, 1 on buggy |
| rb-f5635c27 | f5635c27 | none | none |  |
| rb-d8444163 | d8444163 | wrong_result | wrong_result | stmt 3: 0 rows on fixed, 1 on buggy |
| rb-62e46c00 | 62e46c00 | error | error | stmt 2: 42P10 there is no unique or exclusion constraint matching the ON CONFLICT specific |
| rb-738d19d4 | 738d19d4 | internal_error | internal_error | stmt 6: XX000 unsupported join alias expression |
| rb-6b858a36 | 6b858a36 | assertion | assertion | subselect.c: !IsA(node, SubPlan) |
| rb-8b29b8da | 8b29b8da | internal_error | internal_error | stmt 10: XX000 could not find pathkey item to sort |
| rb-d764d61a | d764d61a | wrong_result | wrong_result | stmt 7: 1 rows on fixed, 1 on buggy |
| rb-dfcf6fa7 | dfcf6fa7 | internal_error | internal_error | stmt 6: XX000 could not find pathkey item to sort |
| rb-a464d31f | a464d31f | assertion | assertion | relnode.c: join_clause_is_movable_into(rinfo, baserel->relids, joinrelids) |
| rb-852fd5b8 | 852fd5b8 | internal_error | internal_error | stmt 5: XX000 no relation entry for relid 3 |
| rb-a8bb1a77 | a8bb1a77 | assertion | assertion | rewriteManip.c: context->new_index != INVALID_VAR |
| rb-9d0adee8 | 9d0adee8 | none | none |  |
| rb-895c1b5f | 895c1b5f | none | none |  |
| rb-35d3f206 | 35d3f206 | internal_error | internal_error | stmt 21: XX000 variable not found in subplan target list |
| rb-1bf74c80 | 1bf74c80 | wrong_result | wrong_result | stmt 4: 4 rows on fixed, 3 on buggy |
| rb-7774249d | 7774249d | wrong_result | wrong_result | stmt 1: 1 rows on fixed, 1 on buggy |
| rb-0d995793 | 0d995793 | wrong_result | wrong_result | stmt 4: 0 rows on fixed, 1 on buggy |
| rb-d6eac691 | d6eac691 | wrong_result | wrong_result | stmt 0: 0 rows on fixed, 2 on buggy |
| rb-3b120b1e | 3b120b1e | missing_error | missing_error | stmt 0: fixed raises 22003 integer out of range |
note: rb-94e1385e raises errors on the fixed server: stmt 1: 2201E cannot take logarithm using base one; stmt 2: 2201E cannot take logarithm using base one
note: rb-41180127 raises errors on the fixed server: stmt 0: 22030 duplicate JSON object key value
note: rb-6b858a36 raises errors on the fixed server: stmt 3: 21000 more than one row returned by a subquery used as an expression
note: rb-3b120b1e raises errors on the fixed server: stmt 0: 22003 integer out of range; stmt 1: 22003 integer out of range; stmt 2: 22003 integer out of range

20/23 bugs observable; copy the symptoms into /home/bhuvi/cs5187-term-project/src/bench/reinjected/bugs.yaml
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$
```

bench verify: fixed vs solo vs injected

In words

- Real bugs, not synthetic mutants
- Three server variants isolate each bug
- 8 wrong results, 5 internal errors, 4 assertions, 3 raised or dropped errors
- Only 9 of 20 visible to a crash-only fuzzer

23 → 20

fixes reverted → observable

9 of 20

visible without an oracle

So what / next: With known bugs we can ask which configuration finds how many, and how fast.

Measure 2: how the campaigns were run

7 arms

Random, Grammar, +Oracles, no-bandit, +LLM fragments, +LLM seeds, +LLM both

5 x 1 h

trials per arm, same seeds, 4 workers, assertions on

2 laptops

RQ1/2/4 on one, RQ3 on the other

26bd08fb

one PostgreSQL master commit for all

```
gow@Gowtham:~/cs5187-term-project/runs/memberB-rq1-llm$ sed -n '/Median (IQR)/,/^$/p' report/report.md
## Median (IQR) per configuration

| Metric | Random/Mutation | Grammar | Grammar+Oracles | Grammar+Oracles, no bandit | +LLM fragments | +LLM seeds | Grammar+Oracles+LLM |
|---|---|---|---|---|---|---|---|
| Line coverage | 22.2% (21.9%-22.2%) | 36.7% (35.9%-36.7%) | 36.2% (36.1%-36.4%) | 36.2% (35.9%-36.4%) | 35.7% (35.7%-35.8%) | 35.6% (35.4%-35.8%) | 35.7% (35.6%-35.8%) |
| Branch coverage | 15.1% (15.0%-15.1%) | 27.1% (26.6%-27.2%) | 26.7% (26.6%-27.0%) | 26.8% (26.8%-26.9%) | 26.5% (26.4%-26.5%) | 26.4% (26.3%-26.5%) | 26.4% (26.4%-26.5%) |
| SQL features | 23 (22-24) | 101 (101-101) | 101 (101-101) | 101 (101-101) | 104 (104-104) | 106 (106-106) | 106 (106-106) |
| SQL feature coverage | 15.5% (14.9%-16.2%) | 68.2% (68.2%-68.2%) | 68.2% (68.2%-68.2%) | 68.2% (68.2%-68.2%) | 68.2% (68.2%-68.2%) | 70.3% (70.3%-70.3%) | 71.6% (71.6%-71.6%) | 71.6% (71.6%-71.6%) |
| Feature pairs | 57 (57-58) | 2,517 (2,515-2,519) | 2,471 (2,462-2,472) | 2,517 (2,515-2,517) | 2,461 (2,450-2,471) | 2,484 (2,476-2,501) | 2,457 (2,439-2,473) |
| Distinct query shapes | 8,407 (7,701-8,885) | 80,477 (79,052-81,785) | 73,157 (57,767-77,330) | 75,216 (71,260-83,772) | 55,056 (49,914-58,934) | 75,506 (68,180-81,487) | 53,454 (50,402-72,506) |
| Distinct plan shapes | 43 (42-45) | 20,279 (20,049-21,354) | 34,313 (30,524-36,909) | 23,997 (21,187-24,869) | 25,972 (25,281-28,017) | 29,274 (28,875-36,256) | 26,375 (24,497-35,385) |
| Metamorphic checks run | - | - | 41,740 (36,348-45,612) | 31,491 (29,585-36,088) | 33,230 (28,932-34,763) | 42,488 (40,151-45,633) | 30,520 (30,273-41,184) |
| Plan-variant runs | 154,440 (134,264-173,100) | 105,372 (101,616-108,018) | 166,718 (145,956-182,410) | 125,192 (118,708-142,328) | 131,438 (115,686-137,914) | 171,456 (162,560-184,904) | 124,416 (122,648-165,884) |
| Parse-valid rate | 33.6% (33.6%-33.7%) | 100.0% (100.0%-100.0%) | 100.0% (100.0%-100.0%) | 100.0% (100.0%-100.0%) | 99.9% (99.8%-99.9%) | 100.0% (100.0%-100.0%) | 99.9% (99.9%-99.9%) |
| Valid-query rate | 26.0% (26.0%-26.0%) | 99.3% (82.6%-99.5%) | 96.3% (96.0%-96.6%) | 99.4% (99.3%-99.4%) | 95.5% (95.4%-96.1%) | 98.3% (98.3%-98.6%) | 95.3% (94.5%-95.7%) |
| Statements/s | 298.4 (260.6-335.8) | 67.2 (65.2-69.9) | 39.9 (38.3-43.8) | 45.0 (43.3-51.8) | 32.5 (29.3-33.6) | 42.8 (42.0-49.1) | 31.0 (30.3-43.3) |
| Test cases | 1,074,240 (938,252-1,209,074) | 71,228 (70,105-72,835) | 58,601 (53,593-66,738) | 64,923 (60,977-72,797) | 48,222 (41,687-50,046) | 63,655 (61,591-69,453) | 46,754 (45,779-62,292) |
| Unique bug candidates | 0 (0-0) | 1 (1-2) | 1 (1-3) | 1 (1-2) | 1 (0-2) | 3 (1-4) | 1 (0-2) |
| Known bugs reproduced | 0 (0-0) | 0 (0-0) | 0 (0-0) | 0 (0-0) | 0 (0-0) | 0 (0-0) | 0 (0-0) |
| Time to first bug (s) | - | 1,514.6 (1,090.7-2,027.7) | 1,956.4 (1,172.0-2,133.0) | 470.9 (211.7-2,184.9) | 280.0 (236.4-1,076.4) | 598.6 (373.7-740.9) | 953.9 (546.2-1,516.8) |
```

report tool output over all trials

Medians, Mann-Whitney U, Vargha-Delaney A12. With 5 trials the smallest possible p is 0.008.

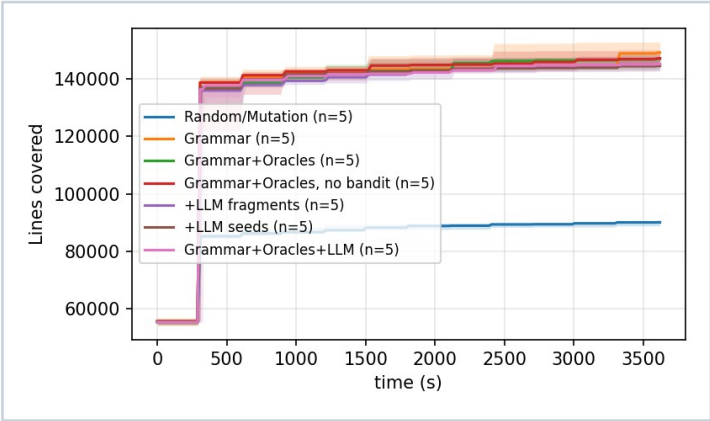
So what / next: The results follow: reach (RQ1), bugs (RQ2), ground truth (RQ3), LLM (RQ4).

Results 1 (RQ1): the grammar reaches far more of PostgreSQL

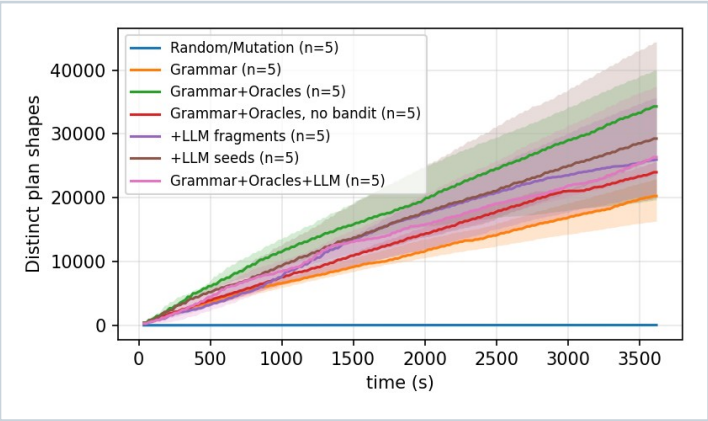
Arm	Valid	Lines	Features	Plan shapes	Tests
Random / mutation	**26.0%**	22.2%	23	43	1,074k
Grammar	**99.3%**	**36.7%**	101	20,279	71k
Grammar + oracles	96.3%	36.2%	101	**34,313**	59k
+ LLM (both)	95.3%	35.7%	106	26,375	47k

Reading

- Grammar vs random: A12 = 1.00, p = 0.008
- Oracles: plan shapes x1.7, no line gain
- LLM: +5 features (p = 0.004), no lines



line coverage over time



distinct plan shapes over time

So what / next: Reach is not bugs: RQ3 and RQ2 ask whether the extra reach finds anything.

Results 2 (RQ3): which arm finds the re-injected bugs

Bug	Random	Grammar	+Oracles	+LLM
dfcf6fa7 (internal error)	0/5	5/5	4/5	5/5
11b56ee2 (assertion)	0/5	0/5	1/5	1/5
8 wrong-result bugs	0	0	0	0
Bugs found of 20	0	1	2	2

```
bhuvi@BlazingBhuvi:~/cs5187-term-project$ cat ~/cs5187-term-project/runs/memberA-rq3/logs/A5_attribute.txt
| bug | symptom | random | grammar | grammar_mm | grammar_llm |
|---|---|---|---|---|---|
| rb-11b56ee2 | assertion | 0/5 | 0/5 | 1/5 (2901s) | 0/5 |
| rb-94e1385e | wrong_result | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-41180127 | missing_error | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-b28a2b2b | wrong_result | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-d8444163 | wrong_result | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-62e46c00 | error | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-738d19d4 | internal_error | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-6b858a36 | assertion | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-8b29b8da | internal_error | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-d764d61a | wrong_result | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-dfcf6fa7 | internal_error | 0/5 | 5/5 (458s) | 4/5 (509s) | 0/5 |
| rb-a464d31f | assertion | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-852fd5b8 | internal_error | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-a8bb1a77 | assertion | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-35d3f206 | internal_error | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-1bf74c80 | wrong_result | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-7774249d | wrong_result | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-0d995793 | wrong_result | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-d6eac691 | wrong_result | 0/5 | 0/5 | 0/5 | 0/5 |
| rb-3b120b1e | missing_error | 0/5 | 0/5 | 0/5 | 0/5 |
| **bugs found** | | **0/20** | **1/20** | **2/20** | **0/20** |

native failures (reproduce on the fixed server): 16; unattributed: 20
bhuvi@BlazingBhuvi:~/cs5187-term-project$
```

How a bug counts

- A finding fails on that bug's solo server and passes on the fixed one
- Otherwise: unattributed

Why so few

- One hour is short for bugs needing a specific feature combination
- 5 trials cannot separate arms that find a bug once in five
- Attribution is conservative: of 30 wrong-result failures, 11 reproduce on the fixed server, 19 on none
- These limits go into the paper

attribution by replay

So what / next: The benchmark shows reach does not yet translate to wrong-result bugs in 1 h; the oracle value shows in RQ2.

Results 3: what the numbers mean

1. Typing pays

- 99% vs 26% valid statements
- 36.7% vs 22.2% of backend lines
- Random runs 15x more tests

2. Oracles find what crashes hide

- Real wrong results in 17/18 (#19793, #19800)
- Cost: precision 0.16
- Ten false-positive classes found and fixed

3. The LLM needs a gate

- +5 features, no coverage
- 4 of 10 commits usable
- A tested rewrite is not a proven rewrite

So what / next: These findings set the plan for the last two weeks.

Evidence and what went wrong

```
Successfully installed contourpy-1.4.0 cycler-0.12.1 fonttools-4.67.0 iniconfig-2.3.1 kiwisolver-1.5.1 matplotlib-3.11.2 num
py-2.5.4 packaging-26.3 pillow-13.0.0 pluggy-1.7.0 psycopg-3.3.6 psycopg-binary-3.3.6 pytest-9.1.1 python-dateutil-3.0.0 sci
py-1.18.1 tzdata-2026.5
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ export OUT=~/cs5187-term-project/runs/memberA-rq3
python3 -m pytest -q 2>&1 | tee $OUT/logs/01_pytest.txt
.....................................................................  [ 52%]
.....................................................................  [100%]
138 passed in 55.06s
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ tail -1 $OUT/logs/01_pytest.txt
138 passed in 55.06s
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$
```

148 unit tests pass

```
10/10 checks passed
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ python3 -m pgfuzz sample configs/rq3.yaml --name grammar_mm -n 3
-- testcase 0 (fresh)
CREATE TABLE n1 (c0 int GENERATED ALWAYS AS IDENTITY, c1 text[]);
INSERT INTO n1 (c1) SELECT string_to_array((x % 100)::text, '') FROM generate_series(1, 5) AS g(x);
ANALYZE n1;
MERGE INTO t3 AS a1 USING (SELECT '8'::text AS c0, CASE (-9223372036854775808)::bigint WHEN MIN ('15'::bigint) THEN CASE 1.5::numeric WHEN 2.25::numeric THEN int4ra
nge(2, 40) WHEN 42.5::numeric THEN '(,5]'::int4range ELSE 'empty'::int4range END WHEN (500)::bigint THEN '[3,)'::int4range ELSE '(,)'::int4range END AS c1 FROM (t3
 AS a2 JOIN tp AS a3 ON ((a2.c0 = a3.c0) AND a2.c1)) WHERE a2.c1 GROUP BY a2.c4 HAVING BOOL_AND (a2.c1) LIMIT 1) AS a4 ON a1.c1 WHEN NOT MATCHED BY SOURCE THEN DELET
E;
SELECT '2024-12-31'::date AS c0, a1.c0 AS c1 FROM t0 AS a1, ROWS FROM (GENERATE_SERIES (1, 3), UNNEST ('{}'::text[])) AS a2 (c0, c1) WHERE TRUE ORDER BY 2 DESC;
-- testcase 1 (fresh)
UPDATE t1 AS a1 SET c0 = COALESCE (a1.c0, 0) WHERE NULLIF (COALESCE ((LEAST ((SELECT JSONB_AGG (((a2.c1 BETWEEN '25'::int AND '45'::int) AND TRUE) ORDER BY ((a2.c1
BETWEEN '25'::int AND '45'::int) AND TRUE)) FROM t1 AS a2 WHERE ('NaN'::text IS NULL)), JSONB_BUILD_OBJECT ('k', LEAST ('{1,2,3,4}'::int[], CASE WHEN TRUE THEN '{}'
::int[] ELSE ('{1,2,3}'::int[] || COALESCE (CASE ARRAY[1, 2, 3] WHEN '{1,2,3}'::int[] THEN a1.c0 WHEN '{}'::int[] THEN a1.c0 ELSE a1.c0 END, 0)) END))) IS NULL), (R
OW ((GREATEST ('{"k": 0, "a": [3, 4, 5]}'::jsonb, '{"k": 1, "a": [1, 2]}'::jsonb) -> 'k'), LEAST ('1 day'::interval, COALESCE ('-1 month'::interval, '7 days'::inter
val))) <> ROW (CASE WHEN FALSE THEN '{"k": -1, "a": [1, 2, 3]}'::jsonb ELSE (SELECT JSON_AGG ('2000-01-01'::date ORDER BY '2000-01-01'::date DESC)::jsonb FROM t4 AS
 a3) END, (CASE WHEN GREATEST (TRUE, TRUE) THEN '00:00:00.000001'::interval ELSE '1 day'::interval END * 0))))), ((REVERSE ('a'::text) ^@ a1.c1) OR COALESCE (((a1.c1
 COLLATE "C") >= GREATEST ('abc'::text, a1.c1)), NULLIF (FALSE, ('{"k": -1, "a": [1, 2, 3]}'::jsonb ? 'x')))));
SELECT FALSE AS c0, CARDINALITY (MIN (REGEXP_SPLIT_TO_ARRAY (a1.c1, '[ab]'))) AS c1, (- 5e-324::float8) AS c2, '00:00:00.000001'::interval AS c3 FROM JSON_TABLE (TO
_JSONB (('-1 month'::interval * 1)), '$[*]' COLUMNS (c0 int PATH '$.k', c1 text PATH '$')) AS a1 GROUP BY a1.c0 ORDER BY 4 NULLS FIRST;
-- testcase 2 (fresh)
SELECT COALESCE (SUBSTR (LOWER (CASE LEAST ('2000-01-01 00:00:00+00'::timestamptz, '2024-01-01 15:00:00+00'::timestamptz) WHEN '1999-12-31 23:59:59.999999+05:30'::t
imestamptz THEN '[1,10)'::text WHEN '2024-01-01 05:00:00+00'::timestamptz THEN JSON_VALUE (CASE WHEN (NOT TRUE) THEN JSON_QUERY (CASE 'x%'::text WHEN FORMAT ('%s|%s
', '(,)'::int4range, (-1)::bigint) THEN CASE '2000-01-01 00:00:00+00'::timestamptz WHEN '1999-12-31 23:59:59.999999+05:30'::timestamptz THEN '{"k": 1, "a": [1, 2]}'
::jsonb WHEN 'epoch'::timestamptz THEN '{"k": 0, "a": [3, 4, 5]}'::jsonb ELSE '{"k": 2, "a": [0]}'::jsonb END WHEN '23x'::text THEN '{"k": 1, "a": [1, 2]}'::jsonb E
LSE '{"k": 2, "a": [0]}'::jsonb END, '$.a[last]') ELSE '{"k": 0, "a": [3, 4, 5]}'::jsonb END, '$.a.size()') ELSE 'A'::text END), NTILE (3) OVER (PARTITION BY JSON_A
RRAY ('6.333333333333333'::float8, '50'::float8 RETURNING jsonb), 1::bigint ORDER BY '350:00:00'::interval, '{"k": 0, "a": [3, 4, 5]}'::jsonb NULLS LAST)), (ABS ('1
2.00'::numeric))::text) AS c0, '2.5'::numeric AS c1 FROM (SELECT FALSE AS c0, a1.c1 AS c1, '2024-01-26'::date AS c2 FROM ti AS a1 GROUP BY GROUPING SETS ((a1.c1), (
))) AS a2 WHERE ('[3,)'::int4range -|- INT4RANGE (LEAST (COALESCE (('25'::int + CARDINALITY (LEAST (ARRAY_REMOVE ('{1,2,3}'::int[], 500), '{}'::int[]))), LENGTH ('A
'::text)), 0), GREATEST ((((SELECT MAX (a3.c0) FROM ti AS a3 WHERE ('%'::text BETWEEN SYMMETRIC a3.c1 AND SUBSTR (REPLACE ('x'::text, a3.c1, a2.c1), '4'::int))) % 1
00) * (GREATEST ((-15), (NULLIF ('50'::int, 60) + 38)) % 100)), 0))) HAVING ('empty'::int4range >> RANGE_INTERSECT_AGG ('[3,)'::int4range));
DELETE FROM t3 AS a1 USING t0 AS a2 WHERE (EXISTS (SELECT a1.c1 AS c0, '15'::bigint AS c1, COUNT (*) AS c2 FROM v0 AS a3 WHERE a1.c1 GROUP BY a3.n HAVING (TRUE OR a
1.c1))) RETURNING a2.c0 AS c0, CASE WHEN CASE INT4RANGE (LEAST (('2024-02-29'::date - a1.c2), 0), GREATEST (GREATEST (a2.c0, 7), 0)) WHEN CASE WHEN a1.c1 THEN INT4R
ANGE (LEAST (a2.c0, 10), GREATEST (a2.c0, 10), '[]') ELSE int4range(2, 40) END THEN a1.c1 WHEN '(,5]'::int4range THEN (NULL::jsonb @> TO_JSONB (ARRAY['1'::text])) E
LSE (COALESCE ('[1,10)'::int4range, '[0,0]'::int4range) &> INT4RANGE (LEAST (ASCII (SPLIT_PART (a2.c1, ',', (-1))), 10), GREATEST (a1.c0, 10), '(]')) END THEN '-0'::
:float8 ELSE (100.0::float8 - '6.333333333333333'::float8) END AS c1, '12.00'::numeric AS c2;
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$
```

grammar sample: generated SQL and validity

Incidents and deviations (all reported in the paper)

- A worker's server failed crash recovery after a BUG-01 assertion: 14 trials re-run with the same seeds
- Laptop sleep: trial clocks are monotonic, budget unchanged
- LLM arm used Gemini Flash (free tier) via the tool's manual mode; 10 of 25 commits because of the daily quota
- 4 false-positive classes fixed after the campaigns, not re-measured
- 5 trials instead of 10 per arm

So what / next: The remaining work is mostly verification and packaging.

Next: plan to 13 November

When	What	Who
30 Oct to 5 Nov	Verify 6 open candidates (5 RLS, 1 corr); upstream follow-up; re-run BUG-01/02 reproducers	Bhuvan, Venu
2 to 6 Nov	Meeting 3: full result tables, paper draft	all
12 Nov	Presentation	all
13 Nov	Paper and repository due	all

Questions for you

- Do committer-confirmed bugs (one fixed) meet the top band, although the benchmark shows no oracle advantage?
- Is it acceptable to argue post-fix precision from reproducers instead of a new campaign?

So what / next: Meeting 3 (2-6 Nov) will show verified open cases and the paper draft.

Thank you. Questions?

github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi